

when selecting the depth and width of a neural net , I notice most common architectures use a monotone direction (ascending or descending width when encoding/compressing data) . so why choose one or the other and why not have a mix of both and forsake monotony all together ?

I was told this affects the geometry of the representation learned , how so ? in proper mathematical entities and terms .

I also wanted to know about Riemannian varieties

This is a very deep question, and it sits exactly at the intersection of **representation learning, differential geometry, and topology** — which fits very well with the kind of work you've been doing.

Let's go layer by layer.

1) Why do many architectures use monotone width?

By monotone width, you mean things like

$$d_0 \rightarrow d_1 \rightarrow d_2 \rightarrow \cdots \rightarrow d_L$$

with either

- **decreasing widths:** $512 \rightarrow 256 \rightarrow 128 \rightarrow 64$
- **increasing widths:** $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$

This is common because it imposes a **geometric prior** on the learned map.

A neural network is a composition of maps:

$$f = f_L \circ f_{L-1} \circ \cdots \circ f_1$$

where each layer is

$$f_\ell(x) = \sigma(W_\ell x + b_\ell)$$

and

$$W_\ell \in \mathbb{R}^{d_{\ell+1} \times d_\ell}$$

The sequence d_ℓ determines the geometry of intermediate spaces.

A) Descending widths = compression / bottleneck geometry

Example:

$$1024 \rightarrow 512 \rightarrow 128 \rightarrow 32$$

This forces the representation through lower-dimensional spaces.

Geometrically this means the network learns a map

$$\mathbb{R}^{1024} \rightarrow \mathbb{R}^{32}$$

through intermediate projections.

This is effectively a **learned dimensionality reduction**.

Very similar in spirit to:

- PCA
- autoencoder bottlenecks
- manifold embeddings

The hidden representation is encouraged to lie on a lower-dimensional manifold.

NeurIPS Papers +2

B) Increasing widths = expansion / feature lifting

Example:

$$64 \rightarrow 256 \rightarrow 1024$$

This is feature lifting.

Instead of compressing, you embed the data into a higher-dimensional feature space:

$$\phi : \mathbb{R}^{64} \rightarrow \mathbb{R}^{1024}$$

This often makes nonlinear separation easier.

This is the same intuition as kernel methods:

$$x \mapsto \phi(x)$$

where classes become linearly separable in high dimension.

2) Why choose one over the other?

The short answer:

- **compress** when you want abstraction / invariance
 - **expand** when you want separability / expressivity
-

Compression encourages invariance

Suppose your input contains nuisance variables:

$$x = (s, n)$$

where

- s : signal
- n : noise

A bottleneck forces the network to preserve only directions useful for the loss.

In information terms, it tends to maximize

$$I(Z; Y)$$

while reducing

$$I(Z; X)$$

This is closely related to the **information bottleneck principle**.

Expansion encourages linear separability

For classification, expanding dimension often helps because of concentration and separation phenomena.

In high dimensions, classes can often be separated by hyperplanes after nonlinear lifting.

This is why transformers and CNN blocks often temporarily increase channels.

3) Why not mix both?

Excellent instinct.

In fact **modern architectures absolutely do mix both**.

Examples:

- ResNet bottleneck blocks:

$$256 \rightarrow 64 \rightarrow 64 \rightarrow 256$$

- Transformer FFN:

$$d \rightarrow 4d \rightarrow d$$

This is exactly non-monotone.

So monotonicity is not a mathematical necessity.

It is mainly:

- design simplicity
- optimization stability
- inductive bias

4) Why does this affect geometry?

This is the most important part.

Your friend is correct.

The widths determine the **rank structure of Jacobians** and therefore the geometry of the learned representation manifold.

Representation manifold

Let data lie on a manifold

$$\mathcal{M} \subset \mathbb{R}^n$$

For example images may lie on a low-dimensional manifold inside pixel space.

Each layer induces

$$f_\ell : \mathcal{M}_{\ell-1} \rightarrow \mathcal{M}_\ell$$

where

$$\mathcal{M}_\ell = f_\ell(\mathcal{M}_{\ell-1})$$

So each layer transforms one manifold into another.

Jacobian rank controls local geometry

The local geometry is determined by the Jacobian

$$J_\ell(x) = \frac{\partial f_\ell}{\partial x}$$

Its rank satisfies

$$\text{rank}(J_\ell) \leq \min(d_\ell, d_{\ell+1})$$

This is the key.

If width decreases:

$$d_{\ell+1} < d_\ell$$

then rank must decrease.

This means dimension can collapse.

Locally the map becomes an **immersion**, **submersion**, or even singular map depending on rank.

This is the rigorous geometric reason.

In differential geometry language

If

$$\text{rank}(J) = k$$

locally the image is a k -dimensional immersed submanifold.

So bottlenecks force

$$\dim \mathcal{M}_{\ell+1} \leq d_{\ell+1}$$

Thus architecture width explicitly bounds manifold dimension.

This is probably what interests you most.

5) Why can non-monotone widths be powerful?

Because geometry is often not globally monotone.

Real representations often need:

1. compress
2. disentangle
3. expand
4. recompress

For example:

$$x \rightarrow z_c \rightarrow z_e \rightarrow y$$

where

- z_c : compressed latent factors
- z_e : expanded interaction features

This is extremely common.

Transformer MLP blocks do exactly this.

The expansion allows nonlinear interactions among compressed factors.

6) Proper mathematical interpretation: pullback metric

Now we enter Riemannian geometry.

≡ ChatGPT ▾ ✦ Get Plus ×

✍️ ⬆️ ...

$$z = f(x)$$

The network induces a metric on input space via pullback:

$$g_x = J_f(x)^T J_f(x)$$

This is the **pullback Riemannian metric**.

This tells you how distances in representation space correspond to distances in input space.

Interpretation

For a small perturbation dx ,

$$ds^2 = dx^T g_x dx$$

If g_x has large eigenvalues in some directions, the network stretches those directions.

If small, it contracts them.

Thus width choices indirectly affect:

- curvature
- anisotropy
- collapse directions
- geodesic distances

This is exactly "geometry of representation".

7) About Riemannian varieties

I think you mean **Riemannian manifolds** (sometimes "varieties" in algebraic geometry language).

A smooth manifold $\mathcal{M}$ becomes Riemannian when each tangent space has an inner product:

$$g_p : T_p \mathcal{M} \times T_p \mathcal{M} \rightarrow \mathbb{R}$$

for every point $p \in \mathcal{M}$

This metric varies smoothly with p .

Example

The sphere

$$S^2 = \{x \in \mathbb{R}^3 : \|x\| = 1\}$$

is a 2D manifold with Riemannian metric induced from Euclidean space.

Distances are geodesics:

great-circle distances.

Neural nets and Riemannian manifolds

This is hugely important.

Hidden representations often empirically lie on low-dimensional manifolds.

The network learns:

$$f : \mathcal{M}_X \rightarrow \mathcal{M}_Z$$

and induces a Riemannian metric

$$g = J^T J$$

This is used in:

- latent space interpolation
- geodesic generative modeling
- Fisher-Rao geometry
- information geometry

Very close to your Fisher metric interests.

Wide/deep tradeoffs also affect representation collapse and bottleneck geometry.

arXiv +3

Given your topology background, I think the next natural topic for you is:

how layer widths affect the curvature and topology (Betti numbers / homology) of learned manifolds

—which connects beautifully to Mapper and GW distances.

If you want, I can derive how bottleneck layers change the induced Riemannian metric $g = J^T J$ and connect it to Fisher information in PyTorch terms.

       Sources

